

Constructor

Constructor = Factory machine.

You input parameters → It builds structured objects.

Technical Explanation

Constructor is a function used with new to:

1. Create object
2. Bind this
3. Link prototype
4. Return object

Why Do We Need Constructors?

Without constructor:

```
const user1 = { name: "Prajwal", age: 22 };  
const user2 = { name: "Amit", age: 24 };  
const user3 = { name: "Neha", age: 21 };
```

You are repeating structure again and again ❌

Not scalable ❌

Hard to maintain ❌

Syntax

```
function User(name, email) {  
  this.name = name;  
  this.email = email;  
}
```

Usage:

```
const u1 = new User("A", "a@mail.com");
```

What Happens Internally When You Use new ?

When you write:

```
const user1 = new User("Prajwal", 22);
```

JavaScript secretly does **4 steps**:

◆ Step 1 — Creates Empty Object

```
const obj = {};
```

◆ Step 2 — Links Prototype

```
obj.__proto__ = User.prototype;
```

This is how object gets access to shared methods.

◆ Step 3 — Calls Constructor with this

```
User.call(obj, "Prajwal", 22);
```

So inside constructor:

```
this === obj;
```

◆ Step 4 — Returns the Object

```
return obj;
```

That becomes user1.

 **Visual Flow**

```
new User("Prajwal",22)
```

↓

Create {}

↓

Bind this → {}

↓

Attach prototype

↓

Execute constructor

↓

Return object

What If You Forget new ?

```
const user = User("Prajwal",22);
```

Then:

this → global object (window)

You just polluted global scope 😬

That's why constructors MUST use new.

Constructor vs Class (Modern Syntax)

This:

```
function User(name) {  
  this.name = name;  
}  
User.prototype.sayHi = function () {};
```

Is EXACTLY SAME as:

```
class User {  
  constructor(name) {  
    this.name = name;
```

```
}  
  
sayHi() {}  
}
```

👉 class is just **syntactic sugar** over constructors.

Constructor is just:

A normal function invoked with `new`
that initializes `this` and links prototype.

If you understand this → you understand **how JS OOP really works**.

What is a Constructor in JavaScript?

Answer:

A constructor is a **function used to create and initialize objects** using the new keyword.

It allows creation of multiple instances with shared structure and behavior.

```
function User(name) {  
  this.name = name;  
}
```

```
const u1 = new User("Prajwal");
```

How Do You Prevent Forgetting new ?

Interviewers love this defensive coding question.

```
function User(name) {  
  if (!(this instanceof User)) {  
    return new User(name);  
  }  
  this.name = name;  
}
```

Now it works even without new.

Can a Constructor Return a Value?

Yes — but only if it returns an **object**.

. What Is instanceof and How Does It Work With Constructors?

Checks prototype chain.

```
const u1 = new User("A");
```

```
console.log(u1 instanceof User); // true
```

Because:

```
u1.__proto__ === User.prototype
```

. Can Arrow Functions Be Used as Constructors?

 No.

```
const User = (name) => {  
  this.name = name;  
};
```

```
new User("A"); // ERROR
```

Arrow functions:

- Do NOT have this
- Do NOT have prototype
- Cannot be used with new

How Do You Achieve Inheritance Using Constructors?

Classic interview question.

```
function Animal(name) {  
  this.name = name;  
}
```

```
Animal.prototype.walk = function() {};
```

```
function Dog(name, breed) {  
  Animal.call(this, name);  
  this.breed = breed;  
}
```

```
Dog.prototype = Object.create(Animal.prototype);  
Dog.prototype.constructor = Dog;
```

`Object.create()` links the child prototype to the parent prototype for method inheritance, and resetting `.constructor` restores the correct constructor reference after overwriting the prototype.

Most Important Concept Interviewers Want

They don't care if you memorize syntax.

They want to see if you understand:

JavaScript does NOT create objects from classes.

It links objects using PROTOTYPES.

If you say this confidently → you stand out.